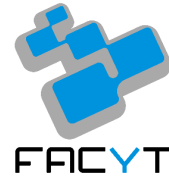




UNIVERSIDAD DE CARABOBO  
FACULTAD EXPERIMENTAL DE CIENCIAS  
Y TECNOLOGÍA  
DEPARTAMENTO DE COMPUTACIÓN  
ARQUITECTURA DEL COMPUTADOR



---

## PRÁCTICA DE LABORATORIO #2

# Implementación y Análisis de Quicksort Iterativo y Mergesort Recursivo en Arquitectura MIPS32

---

**Estudiantes:**

Nathaly Campos *C.I. 32.460.314*

David Serrano *C.I. 31.000.672*

**Profesor:**

José Canache

## 1. ¿Qué diferencias existen entre registros temporales (\$t0-\$t9) y registros guardados (\$s0-\$s7) y cómo se aplicó esta distinción en la práctica?

La convención de llamadas de la arquitectura MIPS32 clasifica los registros de propósito general según la responsabilidad de preservación entre subrutinas:

- **Registros Temporales (\$t0-\$t9):** Son registros de libre modificación por la subrutina llamada (caller-saved). La función llamada puede alterar su contenido sin la obligación de respaldarlo en la memoria.
- **Registros Guardados (\$s0-\$s7):** Son registros cuyo valor debe ser preservado a lo largo de las llamadas a funciones (callee-saved). Si una rutina requiere modificar un registro \$s, está obligada en su prólogo a respaldar su valor en la pila (\$sp) y restaurarlo antes de retornar.

### Aplicación en la práctica:

- En Mergesort Recursivo, los registros \$s0-\$s3 se utilizaron para mantener intactos los límites (left, right, mid) y la dirección del arreglo a través de las constantes invocaciones recursivas, mientras que los registros \$t procesaron los bucles temporales del mezclado.
- En Quicksort Iterativo, los registros temporales (\$t) se usaron de forma intensiva para la lógica de partición interna (esquema de Lomuto, bucle `part_loop`) y el cálculo de direcciones físicas mediante desplazamientos (`sll`). Adicionalmente, se emplearon registros \$s0-\$s2 dentro del ciclo iterativo principal para mantener temporalmente los límites (*low*, *high*) extraídos de la pila auxiliar manual y la posición final del pivote.

## 2. ¿Qué diferencias existen entre los registros \$a0-\$a3, \$v0-\$v1, \$ra y cómo se aplicó esta distinción en la práctica?

Estos registros permiten la comunicación entre rutinas bajo la convención MIPS:

- **Registros de Argumentos (\$a0-\$a3):** Se emplean exclusivamente para transferir parámetros de entrada a las subrutinas.
- **Registros de Valor de Retorno (\$v0-\$v1):** Almacenan los resultados producidos por una subrutina o el código de servicio para llamadas al sistema (`syscall`).
- **Registro de Dirección de Retorno (\$ra):** Guarda de forma automática la dirección de la siguiente instrucción tras un salto y enlace (`jal`).

**Aplicación en la práctica:** En Mergesort Recursivo, `$a0-$a3` pasaron los parámetros de sub-arreglos a las funciones divididas, mientras que el registro `$ra` fue guardado obligatoriamente en la pila en cada nivel recursivo para evitar perder la cadena de retornos. En Quicksort Iterativo, la subrutina `quicksort_iterative` recibió la dirección base y sus límites iniciales en `$a0-$a2`. Dado que toda la lógica de partición se manejó de forma in-line dentro de la misma función, no fue necesario utilizar `$v0` para retornar índices de pivote.

### 3. ¿Cómo afecta el uso de registros frente a memoria en el rendimiento de los algoritmos de ordenamiento implementados?

El acceso a los registros del procesador ocurre en un solo ciclo de reloj durante la etapa de ejecución de la CPU. En contraste, el acceso a la memoria RAM mediante `lw` y `sw` involucra la etapa MEM, introduciendo latencias de bus e incrementando la probabilidad de paradas en el pipeline por dependencias de datos.

En Quicksort Iterativo, el rendimiento se optimizó al mantener el proceso de partición operando fuertemente sobre registros locales, reduciendo las operaciones de memoria (`lw/sw`) solo al momento de desapilar y apilar los rangos de sub-arreglos en la pila auxiliar apuntada por `$t8`. En Mergesort Recursivo, la dependencia de la pila clásica para cada división y el uso del arreglo auxiliar externo imponen un mayor tráfico constante de memoria, haciendo que el rendimiento esté fuertemente condicionado por los tiempos de acceso a la RAM.

### 4. ¿Qué impacto tiene el uso de estructuras de control (bucles anidados, saltos) en la eficiencia de los algoritmos en MIPS32?

Las estructuras de control influyen directamente en la fluidez del pipeline:

- **Bucles anidados:** En la fase de mezcla de Mergesort y en el bucle `part_loop` de Quicksort, las iteraciones repiten comparaciones e incrementos múltiples veces, demandando una ejecución eficiente de instrucciones de salto condicional (`bge`, `bgt`).
- **Riesgos de Control (Control Hazards):** Cada salto incondicional (`j`, `jal`, `jr`) o salto condicional tomado interrumpe la secuencia lineal de captura de instrucciones (Fetch), lo que en procesadores reales obliga a descartar instrucciones en flujo (flush) si no se usan técnicas de predicción de saltos.

### 5. ¿Cuáles son las diferencias de complejidad computacional entre el algoritmo Quicksort y el algoritmo Mergesort? ¿Qué implicaciones tiene esto para la implementación en un entorno

## MIPS32?

- **Quicksort (Iterativo):** Tiene una complejidad temporal promedio de  $\mathcal{O}(n \log n)$  y un peor caso de  $\mathcal{O}(n^2)$ . Su gran ventaja es que ordena *in-place* ( $\mathcal{O}(1)$  espacio auxiliar fuera de la pila manual), lo que ahorra consumo de memoria RAM general.
- **Mergesort (Recursivo):** Garantiza un tiempo  $\mathcal{O}(n \log n)$  en todos los casos (mejor, promedio y peor). Sin embargo, requiere  $\mathcal{O}(n)$  de espacio de memoria adicional en el segmento data para el buffer de mezcla auxiliar y memoria de pila para el árbol de recursión.

**Implicaciones en MIPS32:** Nuestra versión iterativa de Quicksort ahorró tiempo y recursos al no construir múltiples marcos de pila clásicos por llamadas a funciones, manejando la recursividad simulada mediante una única pila plana (reservando 400 bytes y utilizando \$t8 como puntero). Por su parte, Mergesort Recursivo ofrece absoluta predecibilidad en tiempos de ejecución, a costa de un tráfico mucho más pesado en la RAM.

6. ¿Cuáles son las fases del ciclo de ejecución de instrucciones en la arquitectura MIPS32 (camino de datos)? ¿En qué consisten?

El camino de datos (datapath) de MIPS32 consta de 5 etapas fundamentales:

1. **IF (Instruction Fetch):** Búsqueda de la instrucción en la memoria apuntada por el contador de programa (PC) e incremento de  $PC + 4$ .
2. **ID (Instruction Decode / Register Read):** Decodificación del código de operación y lectura de registros fuentes.
3. **EX (Execute / Address Calculation):** Operación en la Unidad Aritmético-Lógica (ALU) para calcular resultados, direcciones de memoria o condiciones de salto.
4. **MEM (Memory Access):** Lectura o escritura en la memoria RAM principal si se ejecuta una instrucción `lw` o `sw`.
5. **WB (Write Back):** Escritura del resultado final de regreso en el registro destino del banco de registros.

7. ¿Qué tipo de instrucciones se usaron predominantemente en la práctica (R, I, J) y por qué?

- **Tipo I (Immediate):** Fueron las más utilizadas. Incluyen accesos a memoria (`lw`, `sw`), saltos de condición (`beq`, `bne`, `bge`) e instrucciones aritméticas con constantes (`addi`, `subi`, `sll`). El acceso a arreglos y el control de ciclos justifican su alta frecuencia.
- **Tipo R (Register):** Empleadas para comparaciones lógicas y operaciones aritméticas directas entre registros (`add`, `sub`, `slt`), así como para retornos de función con `jr $ra`.
- **Tipo J (Jump):** Utilizadas para los saltos incondicionales globales (`j`) en los bucles de Quicksort y llamadas a función (`jal`) en la recursión de Mergesort.

8. ¿Cómo se ve afectado el rendimiento si se abusa del uso de instrucciones de salto (`j`, `beq`, `bne`) en lugar de usar estructuras lineales?

El abuso de instrucciones de salto impacta el rendimiento del procesador MIPS32

debido a:

- Alteración del flujo secuencial en la memoria de instrucciones, reduciendo la efectividad del caché de instrucciones (I-Cache).
- Penalizaciones por bifurcación en la etapa de Fetch, ya que el procesador debe calcular la dirección destino antes de continuar cargando la siguiente instrucción válida.

## 9. ¿Qué ventajas ofrece el modelo RISC de MIPS en la implementación de algoritmos básicos como los de ordenamiento?

El modelo RISC (Reduced Instruction Set Computer) brinda grandes ventajas:

- **Set de instrucciones simplificado e instrucciones de 32 bits fijas:** Facilita el diseño de un hardware con ciclo de reloj sumamente rápido.
- **Arquitectura Load/Store:** Separa estrictamente el acceso a la memoria RAM de los cálculos de la ALU, garantizando que las comparaciones de ordenamiento se hagan a máxima velocidad en los registros.
- **Banco de 32 registros generales:** Permite retener variables de iteración, direcciones e índices sin depender de memoria RAM lenta.

## 10. ¿Cómo se usó el modo de ejecución paso a paso (Step, Step Into) en MARS para verificar la correcta ejecución del algoritmo?

- **Step (F7):** Se utilizó para auditar el avance lineal del bucle interno `part_loop` en Quicksort y el bucle de copia en Mergesort, verificando la correcta actualización de contadores e índices.
- **Step Into:** Fue indispensable en Mergesort Recursivo para ingresar al interior de las funciones invocadas con `jal`, permitiendo comprobar que el Stack Pointer (`$sp`) reservara exactamente el espacio del marco de pila y resguardara la dirección de retorno `$ra` antes de la siguiente división.

### 11. ¿Qué herramienta de MARS fue más útil para observar el contenido de los registros y detectar errores lógicos?

Las herramientas esenciales fueron el panel de **Registers Display** y la ventana de **Data Segment**:

- El panel de registros permitió rastrear la evolución de los valores (`$t8` actuando como puntero de pila en Quicksort, o los límites pasados por `$a0-$a3`).
- La ventana de Data Segment permitió visualizar directamente en la memoria RAM cómo los elementos del arreglo cambian de posición tras cada intercambio en la partición de Quicksort o se vuelcan ordenados desde el arreglo temporal en Mergesort.

### 12. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo R? (por ejemplo: add)

Mediante la herramienta integrada **MARS MIPS X-Ray Display Tool**:

- Se resalta la activación de líneas de control: `RegDst = 1` (selecciona el registro `rd`), `ALUSrc = 0` (el segundo operando proviene del banco de registros), `MemToReg = 0`, `RegWrite = 1` y señales de lectura/escritura de memoria inactivas.
- Permite observar la ruta limpia que conecta los registros fuente con las entradas A y B de la ALU y el posterior retorno del resultado hacia el registro destino.

### 13. ¿Cómo puede visualizarse en MARS el camino de datos para una instrucción tipo I? (por ejemplo: lw)

A través de la misma herramienta **MIPS X-Ray**:

- Se observan las señales de control activas: `ALUSrc = 1` (pasa el valor inmediato extendido en signo a la ALU), `MemRead = 1` (habilita la lectura de la memoria de datos), `MemToReg = 1` (envía el dato de la memoria al registro) y `RegWrite = 1`.
- Destaca el paso del inmediato por la unidad de extensión de signo (Sign Extend), el cálculo de la dirección efectiva (`Base + Offset`) en la ALU y la transferencia del dato leído desde la RAM hacia el registro en el banco.

### 14. Justificar la elección del algoritmo alternativo

La elección de implementar **Quicksort en su forma Iterativa** como contraparte de **Mergesort Recursivo** se justifica plenamente bajo criterios de arquitectura de

computadores:

- Demuestra cómo un algoritmo conceptualmente recursivo puede adaptarse a sistemas embebidos mediante una **pila explícita manual**, eliminando el costo implícito de guardado de contextos de función (stack frames) y previniendo desbordamientos de pila de la CPU.
- Ofrece un contraste didáctico ideal: mientras Mergesort explora el uso tradicional de la pila de llamadas con `jal` y `$ra`, Quicksort Iterativo exhibe un control directo del flujo de datos integrando lógicas in-line en la misma rutina.

## 15. Análisis y Discusión de los Resultados

Tras probar ambos programas en el simulador MARS con arreglos de prueba en distintas condiciones, se obtienen las siguientes conclusiones:

- **Quicksort Iterativo:** Mostró una alta eficiencia al realizar todo el proceso *in-place*. Al integrar la partición internamente y evitar múltiples llamadas a funciones con `jal`, redujo drásticamente el peso de crear contextos de sistema, gestionando los límites directamente a través de una pila auxiliar ligera gobernada por el registro `$t8`.
- **Mergesort Recursivo:** Presentó un comportamiento sumamente estable y garantizado  $\mathcal{O}(n \log n)$ , sin depender del orden previo de los datos. Sin embargo, requirió mayor espacio de memoria (pila de llamadas y buffer auxiliar en data) y una cantidad significativamente mayor de accesos a RAM debido a la fase de copiado en el mezclado.
- **Conclusión Final:** En la arquitectura MIPS32, el diseño de algoritmos debe balancear la elegancia lógica con la realidad física del hardware; integrar particiones iterativas minimiza el tráfico con la memoria de ejecución, ofreciendo un mejor perfil de rendimiento general frente al exceso de llamadas recursivas.